14

IOT GARDEN

We live in an era where our phones talk to light bulbs and where toothbrushes want to access the internet. This is possible through the *Internet of Things (IoT)*, networks of everyday devices embedded with sensors that communicate with each other and the internet, usually in a wireless fashion. In this chapter, you'll build your own IoT sensor network to monitor the temperature and humidity conditions in your garden. The network will consist of one or more low-power devices running Python code and transmitting real-time sensor data wirelessly to a Raspberry Pi. The Pi will log the data and make it available over a local web server. You'll be able to view the sensor data through a web browser, as well as receive real-time alerts on your mobile device when extreme conditions occur.

Some of the concepts you'll learn about through this project are:

- Putting together a low-power IoT sensor network
- Understanding the basics of the Bluetooth Low Energy (BLE) protocol
- Building a BLE scanner on the Raspberry Pi

- Using a SQLite database to store sensor data
- Running a web server on the Pi using `Bottle`
- Using If This Then That (IFTTT) to send alerts to your mobile phone

How It Works

The IoT devices you'll use for this project are Adafruit BLE Sense boards, which have built-in temperature and humidity sensors. The devices periodically take measurements and transmit the sensor data wirelessly using Bluetooth Low Energy (BLE), which we'll discuss soon. This data is picked up by the Raspberry Pi running a BLE scanner. The Pi uses a database to store and retrieve the data, and it also runs a web server so it can display the data on a web page. Additionally, the Pi has logic to detect anomalies in the temperature and humidity data and to send an alert to a user's mobile device via the IFTTT service when such anomalies occur. **Figure 14-1** summarizes the architecture of the project.

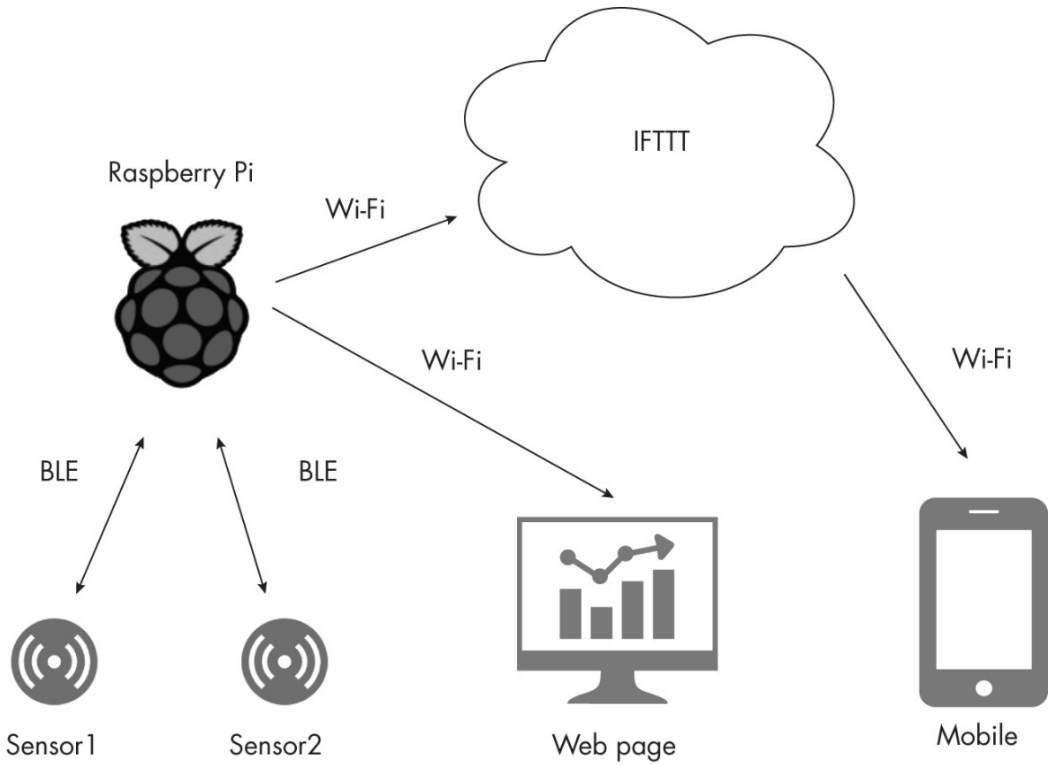

Figure 14-1: *The IoT garden system architecture*

You may wonder why you need the Raspberry Pi at all. Why can't the sensor devices talk directly to the internet instead of going through the Pi? The answer lies in power consumption. If the devices were talking directly to the internet via a protocol like Wi-Fi, they would typically consume more than 10 times the power compared to using BLE. This is important, because IoT devices are usually powered by batteries, and we expect them to last a very long time. The arrangement in **Figure 14-1**, where you have a network of low-power wireless devices talking to a gateway (the Raspberry Pi, in this case), is a common architecture in the world of IoT.

Another feature of this project's architecture is that your data remains in your hands—inside a database on your Pi, to be precise. You aren't blasting it over the internet, but rather confining it to your local network. This may not be critical for basic garden data, but it's still good to know that an IoT device doesn't always *need* to send everything over the internet for it to be useful. Privacy and security are two good reasons for exposing your data to the internet only when it's really necessary. In this case, you'll still make use of the internet a little bit to send the IFTTT alerts.

Bluetooth Low Energy

BLE is a subset of the same wireless technology standard that enables Bluetooth headphones and speakers, but it's optimized for low-power, battery-operated devices. BLE is how your smartphone speaks to your smartwatch or your fitness tracker, for example. Devices that communicate over BLE can be categorized as either *central* or *peripheral*. Usually, central devices are more capable hardware such as laptops and phones, while peripheral devices are less capable hardware such as fitness bands and beacons. In this project, the Raspberry Pi is the central device, and the Adafruit BLE Sense boards are the peripherals.

NOTE *The distinction between central and peripheral devices isn't always clear-cut. Modern BLE chips allow the same piece of hardware to*

function as a central device, a peripheral, or a combination of both.

A BLE peripheral makes its presence known via *advertisement packets*, as shown in **Figure 14-2**. These packets of data, which are typically sent out every few milliseconds, contain information such as the name of the peripheral, its transmission power, its manufacturer data, whether the central device can connect to it, and so on. The central device continuously scans for advertisement packets, and it can use information in the packets to establish communication with the peripheral.

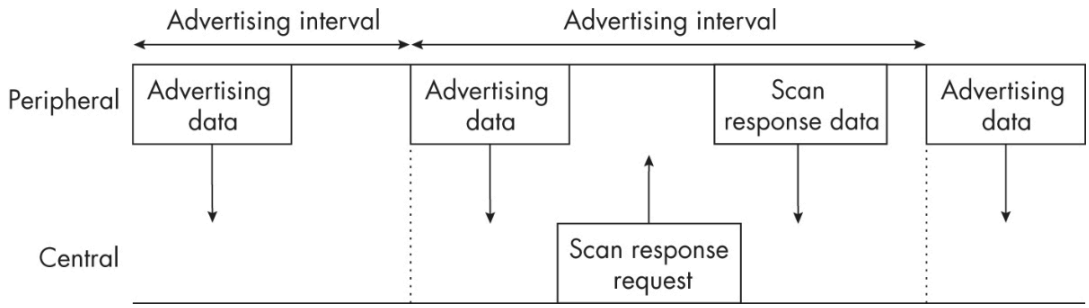

Figure 14-2: *The BLE advertising scheme*

The amount of data in an advertisement packet is limited to just 31 bytes to conserve the peripheral's battery, but the peripheral can optionally send an additional packet of information via a separate transmission called a *scan response*. The peripheral indicates whether a scan response is available as part of its normal advertisement packet. If the central device wants the extra data, it then sends a scan response request, which prompts the peripheral to take a break from sending advertisement packets and send the scan response instead. For this project, though, you need very little data from the sensors, so you can just put the temperature and humidity data directly in the advertisement packets.

On the Raspberry Pi side, you'll build a BLE scanner using BlueZ, the official Bluetooth protocol stack on Linux. Specifically, you'll make use of the following three command line programs:

hciconfig Resets BLE on the Pi during program initialization

hcitool Scans for BLE peripherals

hcidump Reads the advertisement data from the BLE peripherals

`hciconfig` and `hcitool` come as part of the Raspberry Pi OS installation, but you'll need to install `hcidump` from a terminal, as follows:

```
$ sudo apt-get install bluez-hcidump
```

Here's what a typical command line session on the Raspberry Pi looks like with these tools. First, run `hcidump` in a shell to get ready to output data packets once the scan begins:

```
pi@iotsensors:~ $ sudo hcidump --raw

HCI sniffer - Bluetooth packet analyzer ver 5.50

device: hci0 snap_len: 1500 filter: 0xffffffff
```

This tells you that `hcidump` is waiting for BLE input. Next, run the `lescan` command with `hcitool` in a different shell to start scanning for BLE devices:

```
pi@iotsensors:~ $ sudo hcitool lescan

LE Scan...

DE:74:03:D9:3D:8B (unknown)

DE:74:03:D9:3D:8B IOTG1

36:D2:35:5A:BF:B0 (unknown)

8C:79:F5:8C:AE:DA (unknown)
```

```
5D:9F:EC:A0:09:51 (unknown)

5D:9F:EC:A0:09:51 (unknown)

60:80:0A:83:18:40 (unknown)

-- snip --
```

This indicates that the scanner has detected a whole lot of BLE devices (they're everywhere these days!). The moment you run the `lescan` command, `hcidump` starts printing out advertisement packet data, so your `hcidump` shell should now be full of messages like the following:

```
< 01 0B 20 07 01 10 00 10 00 00 00

> 04 0E 04 01 0B 20 00

< 01 0C 20 02 01 01

> 04 0E 04 01 0C 20 00

> 04 3E 1B 02 01 02 01 8B 3D D9 03 74 DE 0F 0E FF 22 08 0A 31

  FE 49 4F 54 47 31 1B 36 30 CB

> 04 3E 16 02 01 04 01 8B 3D D9 03 74 DE 0A 02 0A 00 06 09 49

  4F 54 47 31 CB

> 04 3E 23 02 01 03 01 03 58 0A 00 6A 35 17 16 FF 06 00 01 09

  21 0A 13 71 DA 7D 1A 00 52 6F 63 69 6E 61 6E 74 65 C5

> 04 3E 1F 02 01 03 01 B9 D4 AE 7E 01 0E 13 12 FF 06 00 01 09

  21 0A 9E 54 20 C5 51 48 6D 61 6E 64 6F BE
```

The messages are output as hexadecimal bytes (rather than human-readable text) because you started `hcidump` with the `--raw` option.

This example illustrated using the BlueZ tools manually at the command line. For the project, you'll instead execute the commands from inside the Python code running on your Pi. The Python code will also read the advertisement packets and get the sensor data.

The Bottle Web Framework

To monitor the sensor data via a web interface, you'll need to have the Pi run a web server. To do this, you'll use `Bottle`, a Python web framework with a simple interface. (In fact, the entire library consists of a single source file named *bottle.py*.) Here's the code needed to serve a simple web page from a Pi using `Bottle`:

```
from bottle import route, run

❶ @route('/hello')

def hello():

  ❷ return "Hello Bottle World!"

❸ run(host='iotgarden.local', port=8080, debug=True)
```

This code first defines a route to a URL or path (in this case, `/hello`) where the client can send data requests ❶, and it uses the `route()` method from `Bottle` as a Python decorator to bind to the `hello()` function, which will act as a handler for that route. This way, when the user navigates to the route, `Bottle` will call the `hello()` function, which returns a string ❷. The `run()` method ❸ starts the `Bottle` server, which can now accept connections from clients. Here we're assuming that the server is running on port 8080 on a Pi called `iotgarden`. Notice that the `debug` flag is set to `True` to make it easier to diagnose problems.

Run this code on your Wi-Fi–connected Pi, open a browser on any computer connected to the local network, and visit *http://<iotgarden>.local:8080/hello/*, substituting in your Pi's name as appropriate. `Bottle` should serve you a web page with the text "Hello Bottle World!" With just a few lines of code, you've created a web server.

PYTHON DECORATORS

A *decorator* in Python is `@` syntax that takes a function as an argument (such as `route()` in our `Bottle` example) and returns another function (such as `hello()`). A decorator provides a convenient way to "wrap" one function using another function. For example, this code

```
@wrapper
def myFunc():
    return 'hi'
```

is equivalent to doing the following:

```
myFunc = wrapper(myFunc)
```

Functions are first-class objects in Python that can be passed like variables.

Note that you'll be using `Bottle` routing functions slightly differently in your project compared to this simple example, because you'll be binding the routes to class methods rather than free functions like `hello()` shown previously. There will be more on this later.

The SQLite Database

You need a place to store the sensor data so you can retrieve it in the future. You could write the data to a text file, but the retrieval process would quickly get cumbersome. Instead, you'll store the data with SQLite, a lightweight, easy-to-use database perfect for embedded systems like the Raspberry Pi. To access SQLite in Python, you'll use the `sqlite3` library.

SQLite databases are manipulated using SQL, a standard language for database systems. The SQL statements are written as strings in your Python code. You don't need to be a SQL expert to use SQLite for this project, however. You'll need only a few commands, which we'll discuss as they arise. To get a feel for how it works, let's look at a simple example of using SQLite in a Python interpreter session. First, here's how to create a database and add some entries to it:

```
>>> import sqlite3

>>> con = sqlite3. connect('test.db') ❶

>>> cur = con. cursor() ❷

>>> cur. execute(" CREATE TABLE sensor_data ( TS datetime, ID
text, VAL numeric)") ❸

>>> for i in range(10):

... cur.execute("INSERT into sensor_data VALUES
(datetime('now'),'ABC', ?)", (i, )) ❹

>>> con. commit() ❺

>>> con. close()

>>> exit()
```

Here you call the `sqlite3.connect()` method with the name of the database (`test.db` in this case) ❶. This method either returns a connection to an existing database or creates a new one if the database with the given name doesn't exist. Then you create a *cursor* using the connection object ❷. This is a construct that lets you interact with the database to create tables, make new entries, and retrieve data. You use the cursor to execute a SQL statement that creates a database table called `sensor_data` with the following columns: `TS` (timestamp), which is of the type `datetime`; `ID` of type `text`; and `VAL` of type `numeric` ❸. Next, you add 10 entries to this database by executing SQL `INSERT` statements in a `for` loop. Each statement adds the entry with the current timestamp, the string `'ABC'`, and the loop index `i` ❹. The `?` is a formatting placeholder used by SQLite, and the actual values are specified using a tuple. Finally, you commit the changes to the database to make them permanent ❺, before closing the database connection.

Now let's retrieve some values from the database:

```
>>> con = sqlite3. connect('test.db')

>>> cur = con. cursor()

❶ >>> cur. execute(" SELECT * FROM sensor_data WHERE VAL >
5")

>>> print(cur. fetchall())

[('2021-10-16 13:01:22', 'ABC', 6), ('2021-10-16 13:01:22', 'ABC', 7),

 ('2021-10-16 13:01:22', 'ABC', 8), ('2021-10-16 13:01:22', 'ABC', 9)]
```

Once again, you establish a connection to the database and create a cursor to interact with it. Then you execute a `SELECT` SQL query to retrieve some data ❶. In this query, you ask for all rows (`SELECT *`) from the `sensor_data` table for which the entry in the `VAL` column is

greater than `5`. You print the results of the query, which are accessible through the cursor's `fetchall()` method.

Requirements

On the Raspberry Pi, you'll need the `bottle` module to create a web server, the `sqlite3` module to work with a SQLite database, and `matplotlib` to plot the sensor data. The BLE Sense boards don't have enough computing power to run a full version of Python, so instead you'll program them using CircuitPython, an open source derivative of MicroPython maintained by Adafruit. We're using CircuitPython for this project, rather than MicroPython as you saw in **Chapter 12**, since the former has more library support for the Adafruit-made devices.

You'll also need the following hardware for this project:

- One or more Adafruit Feather Bluefruit nRF52840 Sense boards, as per your need
- One Raspberry Pi 3B+ (or newer) board with SD card and power supply adapter
- One 3.7 V LiPo battery or USB power supply per BLE Sense board

Figure 14-3 shows the required hardware.

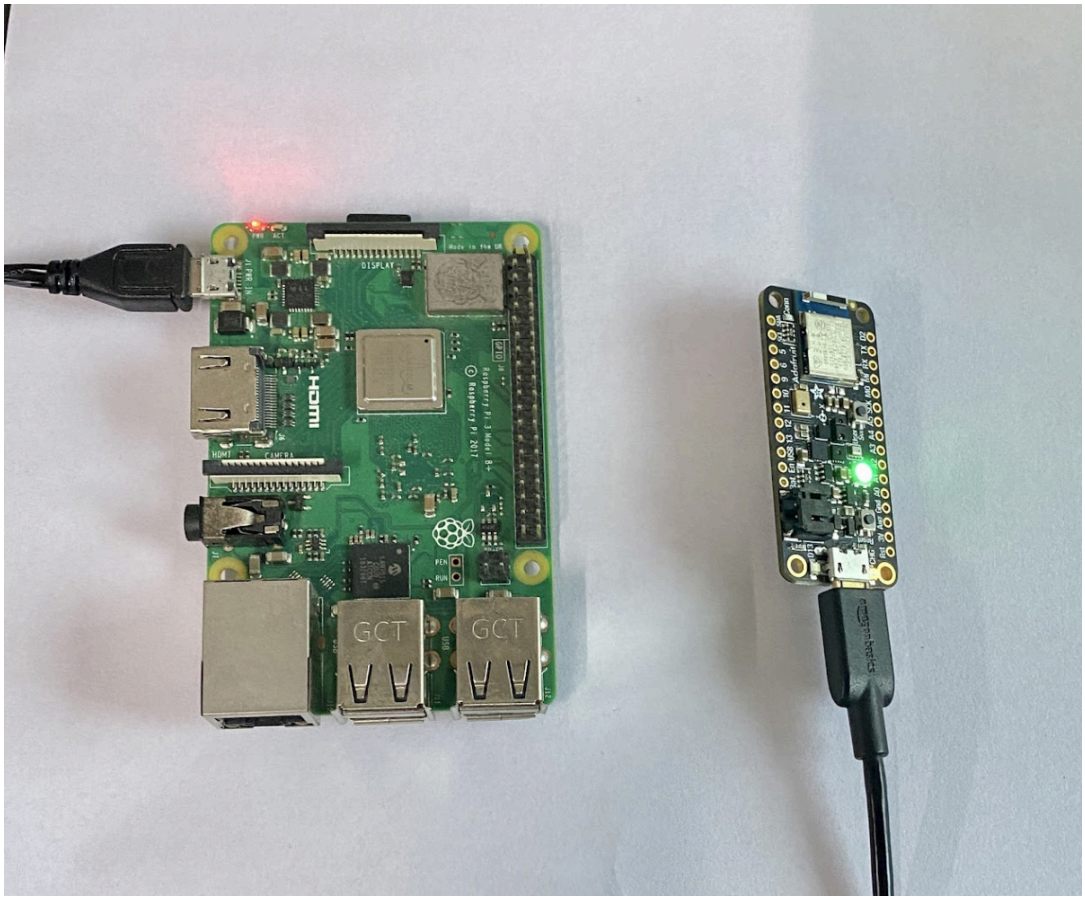

Figure 14-3: *The hardware required for the project*

You can hook up your Raspberry Pi indoors, close to your garden, and place your BLE Sense boards with suitable power supply units and protective enclosures in the garden.

Raspberry Pi Setup

To start this project, you'll need to set up your Raspberry Pi. Please follow the instructions in **Appendix B**. The project code that follows assumes that you've named you Pi `iotgarden`, which allows you to access it on the network as `iotgarden.local`.

CircuitPython Setup

To install CircuitPython, follow these steps for each of your Adafruit BLE Sense boards:

1. Visit ***https://circuitpython.org/downloads***, search for your Bluefruit Sense board, and download the board's CircuitPython installer file, which has a *.uf2* extension. Take note of the CircuitPython version number you're downloading.
2. Connect the Adafuit board to a USB port on your computer and double-click the Reset button on the board. The LED on the board should turn green, and you should see a new drive appear on your computer called FTHRSNSBOOT.
3. Drag the *.uf2* file into the FTHRSNSBOOT drive. Once the file is copied, the LED on the board will flash, and a new drive called CIRCUITPY will appear on your computer.

Next, you need to install the required Adafruit libraries on your board. Here's how:

1. Visit ***https://circuitpython.org/libraries*** and download the *.zip* file with the library bundle corresponding to your version of CircuitPython.
2. 2. Unzip the downloaded file and copy the following files/folders into a folder called *lib* inside the CIRCUITPY drive. (Create the *lib* folder if it doesn't exist.)
    1. *adafruit_apds9960*
    2. *adafruit_ble*
    3. *adafruit_bme280*
    4. *adafruit_bmp280.mpy*
    5. *adafruit_bus_device*
    6. *adafruit_lis3mdl.mpy*
    7. *adafruit_lsm6ds*
    8. *adafruit_register*
    9. *adafruit_sht31d.mpy*
    10. *neopixel.mpy*
3. Press Reset on the board and you're all set to use the board for this project.

By default, CircuitPython will run code from any file in CIRCUITPY named *code.py*. You'll need to copy to the drive the *ble_sensors.py* file discussed in the following section and rename it *code.py* to run the project.

If This Then That Setup

IFTTT is a web service that lets you create automated responses to specific actions. You'll use IFTTT to send alerts to your mobile phone when something is really off with the temperature or humidity levels your sensors are picking up. Follow these steps to get set up to receive IFTTT alerts:

1. Visit the IFTTT website (***https://ifttt.com***) and sign up for an account.
2. Download the IFTTT app to your smartphone and set it up.
3. While signed into your IFTTT account in your browser, click **Create**. Then click the **Add** button in the If This box.
4. Search for and select **Webhooks** on the Choose a Service page that comes up. Then select **Receive a Web Request with a JSON Payload**.
5. Under Event Name, enter **TH_alert** (note the capitalization) and press **Create Trigger**.
6. You should now be back on the Create page. Click the **Add** button in the Then That box.
7. Search for and select **Notifications**. Then click **Send a Notification from the IFTTT App**.
8. In the page that comes up, add the text "T/H Alert!" to the Message box. Then click **Add Ingredient** and select **OccuredAt**. Click **Add Ingredient** again and select **JsonPayload**.
9. Click the **Create Action** button to return to the Create screen. Then click **Continue** and **Finish** to finalize the alert.

You'll need your IFTTT key to trigger alerts from your Python code. To look it up, follow these steps:

1. On the IFTTT website, click the round account icon in the top-right corner of the screen and select **My Services**.
2. Click the **Webhooks** link, and then click the **Documentation** button.
3. A page should load with your key written across the top. Take note of the key. You'll also find information on this page about how to send a test alert to your smartphone. Be sure to fill in **TH_alert** as the event name if you run a test.

The Code

The code for this project is spread over these Python files:

ble_sensors.py The CircuitPython code that runs in the Adafruit BLE Sense boards. It reads from the temperature and humidity sensors and puts the data in BLE advertisement packets.

BLEScanner.py Implements the BLE scanner on the Raspberry Pi, using BlueZ tools to read advertisement data. This code also sends the IFTTT alerts.

server.py Implements the `Bottle` web server on the Raspberry Pi.

iotgarden.py The main program file. This code sets up the SQLite database and starts the BLE scanner and the web server.

In addition to the Python files, the project has a subfolder called *static* with some extra files used by the `Bottle` web server:

static/style.css The style sheet for the HTML returned by the server

static/server.js The JavaScript code returned by the server that fetches sensor data

The full code for the project is available at ***https://github.com/mkvenkit/pp2e/tree/main/iotgarden***.

The CircuitPython Code

The CircuitPython code running on the BLE Sense board has a straightforward purpose: it reads data from the built-in temperature and humidity sensors and puts that data in the board's BLE advertisement packets. This deceptively simple task requires you to import a surprising number of modules. To see the complete code listing, skip ahead to **"The Complete CircuitPython Code"** on **page 343**. The code is also available at ***https://github.com/mkvenkit/pp2e/blob/main/iotgarden/ble_sensors/ble_sensors.py***.

```
import time, struct

import board

import adafruit_bmp280

import adafruit_sht31d

from adafruit_ble import BLERadio

❶ from adafruit_ble.advertising import Advertisement,
LazyObjectField

❷ from adafruit_ble.advertising.standard import ManufacturerData,
                                                ManufacturerDataField

import _bleio

import neopixel
```

You import Python's built-in `time` and `struct` modules for sleeping and packing data, respectively. The `board` module gives you access to the `I2C` library, which allows the BLE chip on the board to communicate with the sensors using the I^2C protocol (pronounced "eye-

squared-C"). The `adafruit_bmp280` and `adafruit_sht31d` modules are required for communicating with the sensors, and the `BLERadio` class is for enabling BLE and sending advertisement packets. The imports at ❶ and ❷ give you access to the Adafruit BLE advertising modules necessary for creating your own custom advertisement packets featuring the sensor data. Additionally, you'll use the `_bleio` module to get the MAC address of the device and `neopixel` to control the board's LED.

Preparing BLE Packets

Next, define a class called `IOTGAdvertisement` to help create the BLE advertisement packets. The `adafruit_ble` library already has an `Advertisement` class that handles BLE advertisements. You create `IOTGAdvertisement` as a subclass of `Advertisement` to use the parent class's features while adding your own customization:

```
class IOTGAdvertisement(Advertisement):
    flags = None
❶   match_prefixes = (
        struct.pack(
            "<BHBH",  # prefix format
            0xFF,     # 0xFF is "Manufacturer Specific Data" as per BLE spec
            0x0822,   # 2-byte company ID
            struct.calcsize("<H9s"), # data format
            0xabcd # our ID
        ), # comma required - a tuple is expected
```

```
    )

❷ manufacturer_data = LazyObjectField(
        ManufacturerData,
        "manufacturer_data",
        advertising_data_type=0xFF, # 0xFF is "Manufacturer Specific
Data"
                                    # as per BLE spec
        company_id=0x0822,          # 2-byte company ID
        key_encoding="<H",
    )

    # set manufacturer data field

❸ md_field = ManufacturerDataField(0xabcd, "<9s")
```

The BLE standard is very particular, so this code may look intricate, but all it's really doing is putting some custom data in the advertisement packet. First you fill a tuple called `match_prefixes` ❶, which the `adafruit_ble` library will use to manage various fields in the advertisement packet. The tuple has only one element, a packed structure of bytes that you create using the Python `struct` module. Next, you define the `manufacturer_data` field ❷, which will use the format described at ❶. The manufacturer data field is a standard part of a BLE advertisement packet that has some space for whatever custom data the manufacturer (or the user) wants to include. Finally, you create a custom `ManufacturerDataField` object ❸, which you'll keep updating as sensor values change.

Reading and Sending Data

The `main()` function of the CircuitPython program reads and sends the sensor data. The function begins with some initializations:

```
def main():
    # initialize I2C
❶ i2c = board.I2C()
    # initialize sensors
❷ bmp280 = adafruit_bmp280.Adafruit_BMP280_I2C(i2c)
❸ sht31d = adafruit_sht31d.SHT31D(i2c)
    # initialize BLE
❹ ble = BLERadio()
    # create custom advertisement object
❺ advertisement = IOTGAdvertisement()
    # append first 2 hex bytes (4 characters) of MAC address to name
❻ addr_bytes = _bleio.adapter.address.address_bytes
    name = "{0:02x}{1:02x}".format(addr_bytes[5],
addr_bytes[4]).upper()
    # set device name
❼ ble.name = "IG" + name
```

First you initialize the `I2C` module ❶ so the BLE chip can communicate with the sensors. Then you initialize the modules for the temperature (`bmp280`) ❷ and humidity (`sht31d`) ❸ sensors. You also initialize the BLE radio ❹, which is required for transmitting the advertisement packets, and create an instance of your custom `IOTGAdvertisement` class ❺.

Next, you set the name of the BLE device to the string `IG` (for IoT Garden) followed by the first four hexadecimal digits (or two bytes) of the device's MAC address ❼. For example, if the MAC address of the device is `de:74:03:d9:3d:8b`, the name of the device will be set to `IGDE74`. To do this, you first get the MAC address as bytes ❻. The bytes are in reverse order of the string representation, however—in our example MAC address, for instance, the first byte would be `0x8b`. What you're looking for are the first two bytes, `0xde` and `0x74`, which are at indices `5` and `4` in `address_bytes`, respectively. You use string formatting to convert these bytes to string representation and convert them to uppercase using `upper()`.

Now let's look at the rest of the initialization:

```
# set initial value

# will use only first 5 chars of name

❶ advertisement.md_field = ble.name[:5] + "0000"

# BLE advertising interval in seconds

BLE_ADV_INT = 0.2

# start BLE advertising

❷ ble.start_advertising(advertisement, interval=BLE_ADV_INT)

# set up NeoPixels and turn them all off
```

```
pixels = neopixel.NeoPixel(board.NEOPIXEL, 1,

                           brightness=0.1, auto_write=False)
```

Here you set an initial value for your custom manufacturer data ❶. For this, you concatenate the first five characters of the device name followed by four bytes of zeros. You'll update the first two bytes with the sensor data. As for the remaining two bytes, there's an exercise waiting for you in **"Experiments!"** on **page 343** where you can put them to use.

Next, you set the BLE advertisement interval (`BLE_ADV_INT`) to `0.2`, meaning the device will send out an advertisement packet every 0.2 seconds. Then you call the method to start sending advertisement packets ❷, passing your custom advertisement class and the time interval as arguments. You also initialize the `neopixel` library to control the LED on the board. The `board.NEOPIXEL` argument sets the pin number for the neopixel LED, and `1` represents the number of LEDs on the board. The brightness setting is `0.1` (the maximum being `1.0`), and setting the `auto_write` flag to `False` means you'll need to call the `show()` method explicitly for the values to take. (You'll see this in action soon.)

The `main()` function continues with a loop that reads the sensor data and updates the BLE packets:

```
# main loop

while True:

    # print values - this will be available on serial

❶ print("Temperature: {:.1f} C".format(bmp280.temperature))

    print("Humidity: {:.1f} %".format(sht31d.relative_humidity))
```

```
    # get sensor data
❷ T = int(bmp280.temperature)+ 40
    H = int(sht31d.relative_humidity)
    # stop advertising
❸ ble.stop_advertising()
    # update advertisement data
❹ advertisement.md_field = ble.name[:5] + chr(T) + chr(H) + "00"
    # start advertising
❺ ble.start_advertising(advertisement, interval=BLE_ADV_INT)
    # blink neopixel LED
    pixels.fill((255, 255, 0))
    pixels.show()
    time.sleep(0.1)
    pixels.fill((0, 0, 0))
    pixels.show()
    # sleep for 2 seconds
❻ time.sleep(2)
```

You begin the loop by printing out the values read by the sensors ❶. You can use this output to confirm that your sensors are putting out reasonable values. To see the values, connect the board via USB to

your computer and use a serial terminal application such as CoolTerm, as explained in **Chapter 12**. The output should look something like this:

```
Temperature: 26.7 C

Humidity: 55.6 %
```

Next, you read the temperature and humidity values from the sensors ❷, converting both to integers since you have only one byte to represent each value. You add 40 to the temperature to accommodate negative values. The BMP280 temperature sensor on the board has a range of −40°C to 85°C, so adding 40 converts this range to [0, 125]. You'll be back to the correct range on the Raspberry Pi once the data is parsed from the BLE advertisement data.

You have to stop BLE advertising ❸ so you can change the data in the packets. Then you set the manufacturer data field with the first five characters of the device name, followed by one byte each of temperature and humidity values ❹. You're using `chr()` here to encode each 1-byte value into a character. You also set the last two bytes in the data field to be zeros. Now that you've updated the sensor values in the packet, you restart advertising ❺. This way, the scanner on the Pi will pick up the new sensor values from the BLE board.

To provide a visual indicator that the device is alive, you blink its neopixel LED by turning it on for 0.1 second. The `fill()` method sets a color using an (R, G, B) tuple, and `show()` sets the value to the LED. Finally, you add a two-second delay before restarting the loop and checking the sensors again ❻. During that delay, the board will continue sending out the same advertisement packet every 0.2 seconds, as per its advertisement interval.

NOTE *When you've tested the code and are ready to deploy your IoT device, I recommend commenting out the `print()` statements and the*

neopixel code to conserve power. Remember, BLE is all about low energy!

The BLE Scanner Code

The code to have your Raspberry Pi listen for and process sensor data over BLE is encapsulated in a class called `BLEScanner`. To see the complete code listing, skip ahead to **"The Complete BLE Scanner Code"** on **page 345**. You'll also find this code in the book's GitHub repository at ***https://github.com/mkvenkit/pp2e/blob/main/iotgarden/BLEScanner.py***.

Here's the constructor for this class:

```
class BLEScanner:

    def __init__(self, dbname):

        """BLEScanner constructor"""

        self.T = 0

        self.H = 0

        # max values

        self.TMAX = 30

        self.HMIN = 20

        # timestamp for last alert

    ❶ self.last_alert = time.time()

        # alert interval in seconds
```

```
❷ self.ALERT_INT = 60
  # scan interval in seconds
❸ self.SCAN_INT = 10
❹ self._dbname = dbname
❺ self.hcitool = None
  self.hcidump = None
❻ self.task = None
  # ----------------------------------------
  # peripheral allow list - add your devices here!
  # ----------------------------------------
❼ self.allowlist = ["DE:74:03:D9:3D:8B"]
```

You start by defining instance variables `T` and `H` to keep track of the latest temperature and humidity values read from the sensors. Then you set threshold values for triggering the automated alerts. If the temperature goes above `TMAX` or the humidity goes below `HMIN`, the program will issue an alert using IFTTT. You create the `last_alert` variable at ❶ to store the time when the most recent alert was sent, and at ❷ you set the minimum interval between alerts. This is so you don't keep sending yourself alerts continuously when an alert condition is met. At ❸, you set the scan interval in seconds to control how often the Pi will scan for BLE devices. Next, you save the name of the SQLite database that was passed into the constructor ❹. You need this to save values from the sensors.

At ❺ and the following line, you create a couple of instance variables to store process IDs for the `hcitool` and `hcidump` programs, which will run later. At ❻, you create a `task` instance variable. Later, you'll be creating a separate thread to run this task, which will be doing the scanning, while the main thread of the program runs the web server. Finally, at ❼ you create a list of the BLE devices you want to listen to. When you run the BLE scanner, you're likely to pick up many BLE peripherals, not just the Adafruit Sense boards. Keeping a list with your sensor device IDs makes the BLE data easier to parse. Later in the chapter, I'll show you how to look up the device IDs so you can add them here to your allowlist.

Working with the BlueZ Tools

As we discussed in **"Bluetooth Low Energy"** on **page 313**, this project uses BlueZ, Linux's official Bluetooth protocol stack, to scan for BLE data. The `BLEScanner` class needs methods for working with these tools. First we'll examine the `start_scan()` method, which sets up the BlueZ tools for BLE scanning.

```
def start_scan(self):

    """starts the BlueZ tools required for scanning"""

    print("BLE scan started...")

    # reset device

❶   ret = subprocess.run(['sudo', '-n', 'hciconfig', 'hci0', 'reset'],

                             stdout=subprocess.DEVNULL)

    print(ret)

    # start hcitool process
```

```
❷ self.hcitool = subprocess.Popen(['sudo', '-n', 'hcitool',
                                   'lescan', '--duplicates'],
                                  stdout=subprocess.DEVNULL)
  # start hcidump process
❸ self.hcidump = subprocess.Popen(['sudo', '-n', 'hcidump', '--raw'],
                                  stdout=subprocess.PIPE)
```

First you reset the BLE device of the Raspberry Pi using the `hciconfig` tool ❶. You use the Python `subprocess` module to run this process. The `subprocess.run()` method takes the process arguments as a list, so this call executes the command `sudo -n hciconfig hci0 reset`. The output of this process `stdout` is set to `DEVNULL`, which just means you don't care about messages printed out by this command. (The `-n` flag in `sudo` makes it noninteractive.) You next use a different `subprocess` method called `Popen()` to run the command `sudo -n hcitool lescan --duplicates` ❷. This process scans for BLE peripheral devices. The `--duplicates` flag ensures that the same device can come up in the scan list repeatedly. You need this, since the sensor data in the advertisement packets keeps changing, and you need the latest values.

NOTE *The difference between `subprocess.run()` and `subprocess.Popen()` is that the former waits for the process to complete, whereas the latter returns immediately while the process runs in the background.*

Finally, you use `subprocess.Popen()` to run another command: `sudo -n hcidump --raw` ❸. As we discussed earlier in the chapter, this command intercepts and prints out the advertisement data as hexadecimal bytes. Notice that `stdout` is set to `subprocess.PIPE` in this case. This means you can read the output from this process similar to how

you read the contents of a file. More on this in **“Parsing the Data”** below.

Now let’s look at the `stop_scan()` method, which kills the processes begun in the `start_scan()` method when you’re ready to stop scanning for BLE packets.

```
def stop_scan(self):
    """stops BLE scan by killing BlueZ tools processes"""
    subprocess.run(['sudo', 'kill', str(self.hcidump.pid), '-s', 'SIGINT'])
    subprocess.run(['sudo', '-n', 'kill', str(self.hcitool.pid),
                    '-s', "SIGINT"])
    print("BLE scan stopped.")
```

Here you kill off the `hcidump` and `hcitool` processes using `pid`, their process IDs. The command `sudo -n kill pid -s SIGINT` kills a process with the given `pid` by sending it the `SIGINT` interrupt signal.

Parsing the Data

The scanner needs methods for parsing the BLE data it receives. First we’ll consider the `parse_hcidump()` method, which parses the output from the `hcidump` process:

```
def parse_hcidump(self):
    data = ""
    (macid, name, T, H) = (None, None, None, None)
    while True:
```

```
❶ line = self.hcidump.stdout.readline()
❷ line = line.decode()
 if line.startswith('> '):
    data = line[2:]
 elif line.startswith('< '):
    data = ""
 else:
    if data:
      ❸ data += line
      ❹ data = " ".join(data.split())
      ❺ fields = self.parse_data(data)
          success = False
        ❻ try:
            macid = fields["macid"]
            T = fields["T"]
            H = fields["H"]
            name = fields["name"]
            success = True
          except KeyError:
```

```
            # skip this error, since this indicates

            # invalid data

        ❼ pass

    if success:

        ❽ return (macid, name, T, H)
```

Within a `while` loop, you start reading one line at a time from `self.hcidump.stdout` using the `readline()` method ❶, much like you'd read lines from a file. To understand the code that follows, it helps to know a bit about the data being parsed. This is what a typical output from `hcidump` looks like:

```
> 04 3E 1B 02 01 02 01 8B 3D D9 03 74 DE 0F 0E FF 22 08 0A 31

  FE 49 4F 54 47 31 1B 36 30 CB
```

The output is split across two lines, and it starts with a > character. You want to take these two lines and combine them to get a single string such as `"04 3E 1B 02 01 02 01 8B 3D D9 03 74 DE 0F 0E FF 22 08 0A 31 FE 49 4F 54 47 31 1B 36 30 CB"`. To do this, you convert the bytes output from `readline()` to a string using the `decode()` method ❷. Then you use some logic to build up the final string that you want. If the line starts with `>`, you know it's the first of the set of two lines making up an `hcidump` entry, so you store the line and go on to the next one. Lines that start with `<` are ignored. If a line starts with neither `>` nor `<`, then it's the second line of the advertisement, and you join the lines together ❸.

The resulting data will have newline characters in the middle and the end. You get rid of those using a combination of the `split()` and `join()` string methods ❹. This example illustrates how the scheme works:

```
>>> x = "ab cd\n ef\tff\r\n"

>>> x.split()

['ab', 'cd', 'ef', 'ff']

>>> " ".join(x.split())

'ab cd ef ff'
```

Notice from the first line of output how the `split()` method automatically splits up the string at the whitespace characters, producing a list of substrings and removing the whitespace characters in the process. This gets rid of the unwanted newline characters, but it also gets rid of the spaces, which you want to keep. That's where the `join()` method comes in. It merges the list items back into a single string, with a space between each substring, as you can see in the second output line.

Returning to the `parse_hcidump()` method, you now have a complete BLE advertisement packet stored as a string in variable `data`. You call the `parse_data()` method on this string to get the device details in the form of a `fields` dictionary ❺. We'll look at this method soon. Then you retrieve the MAC ID, name, temperature, and humidity values from the dictionary. This code is enclosed in a `try` block ❻ in case the data isn't what you expect. In that case, an exception will be thrown, and you skip that advertisement packet by calling `pass` ❼. If you successfully retrieve all the values, they're returned as a tuple ❽.

Now let's take a look at the `parse_data()` method you used in `parse_hcidump()` to build the `fields` dictionary:

```
def parse_data(self, data):

    fields = {}

    # parse MACID
```

```
❶ x = [int(val, 16) for val in data.split()]

❷ macid = ":".join([format(val, '02x').upper() for val in x[7:13][::-1]])

  # check if MACID is in allowlist

❸ if macid in self.allowlist:

    # look at 6th byte to see PDU type

  ❹ if (x[5] == 0x02): # ADV_IND

    ❺ fields["macid"] = macid

      # parse data

    ❻ fields["T"] = x[26]

      fields["H"] = x[27]

    ❼ name = "".join([format(val, '02x').upper() for val in x[21:26]])

    ❽ name = bytearray.fromhex(name).decode()

      fields["name"] = name

  return fields
```

You start by defining an empty dictionary `fields`, where you'll store the parsed data. Then you split the data into a list of hexadecimal values ❶ and extract the MAC ID of the peripheral that sent the advertisement packet ❷. Here's a sample run of these statements to illustrate how they work:

```
>>> data = "04 3E 1C 02 01 02 01 8B 3D D9 03 74 DE 10 0F FF
22
```

```
            08 0B 31 FE 49 4F 54 47 31 61 62 63 64 BD"
```

```
>>> x = [int(val, 16) for val in data.split()]
```

```
>>> x
```

```
[4, 62, 28, 2, 1, 2, 1, 139, 61, 217, 3, 116, 222, 16, 15, 255, 34,
 8, 11, 49, 254, 73, 79, 84, 71, 49, 97, 98, 99, 100, 189]
```

```
>>> x[7:13][::-1]
```

```
[222, 116, 3, 217, 61, 139]
```

```
>>> [format(val, '02x').upper() for val in x[7:13][::-1]]
```

```
['DE', '74', '03', 'D9', '3D', '8B']
```

```
>>> ":".join([format(val, '02x').upper() for val in x[7:13]
[::-1]])
```

```
'DE:74:03:D9:3D:8B'
```

Notice how the data is first split into a list of decimal numbers (`x`). Then you use a list comprehension and `format()` to create two-character string representations of the numbers, taking just bytes 7 through 12 to extract the MAC ID. The `[::-1]` reverses the MAC ID, since it comes in the opposite order in the packet data. Finally, the `join()` method merges the bytes making up the MAC ID into a single string, using colons as separators.

Continuing with `parse_data()`, you check if the extracted MAC ID matches a device on your allowlist ❸. If not, you ignore the data. Then you check the fifth byte in the data to ensure that the packet type is `ADV_IND` ❹. This ensures that the data is a regular advertisement packet, and not a scan response. Next, you store the MAC ID in the

`fields` dictionary ❺, along with the temperature and humidity values ❻, which you read from the appropriate indices in the data list. You also read the five-character device name you put in the BLE packet ❼, similar to how you read the MAC ID. Then you call `decode()` to convert the bytes to a string ❽. Finally, you return the `fields` dictionary to the caller.

Sending Alerts

Once you've parsed the data from a BLE advertisement packet, you need to send an IFTTT alert if the sensor readings are concerning. Define a `send_alert()` method for this purpose:

```
def send_alert(self):
    # check T, H
  ❶ delta = time.time() - self.last_alert
  ❷ if ((self.T > self.TMAX) or (self.H < self.HMIN)) and
                          (delta > self.ALERT_INT):
        print("Triggering IFTTT alert!")
      ❸ key = 'ABCDEF' # USE YOUR KEY HERE!
      ❹ url = 'https://maker.ifttt.com/trigger/TH_alert/json/with/key/' +
key
        json_data = {"T": self.T, "H": self.H}
      ❺ r = requests.post(url, data = json_data)
        # save last alert
```

```
        self.last_alert = time.time()
```

First you compute how much time has elapsed since the last time an IFTTT alert was sent ❶. Then you check your alert conditions ❷. An alert will be triggered either if the current temperature is above `TMAX` or if the humidity has fallen below `HMIN`, provided a sufficient amount of time has passed since the last alert. Since the goal is to monitor the health of your garden, checking if it's too hot or too dry makes sense, but you're welcome to modify this check with your own alert criteria. You next put together the IFTTT Webhooks URL ❹, using your user key set at ❸. (Make sure your key matches the one you obtained in **"If This Then That Setup" on page 319**.) Then you set up a simple JSON string with the sensor data and post it to the IFTTT URL ❺. You finish by updating the `last_alert` instance variable with the current time, for future use.

Conducting a Scan

The `scan_task()` method coordinates all the activity required to conduct a BLE scan. This method also stores the scanned data to the SQLite database. Here's the method definition:

```
def scan_task(self):

    """the scanning task which is run on a separate thread"""

    # start BLE scan

❶   self.start_scan()

    # get data

❷   (macid, name, self.T, self.H) = self.parse_hcidump()

    # correct temperature offset
```

```
        self.T = self.T - 40
        print(self.T, self.H)
        # stop BLE scan
    ❸ self.stop_scan()
        # send alert if required
    ❹ self.send_alert()
        # write to db
        # connect to database
        con = sqlite3.connect(self._dbname)
        cur = con.cursor()
        devID = macid
        # add data
        with con:
          ❺ cur.execute("INSERT INTO iotgarden_data VALUES (?, ?, ?, ?, ?)",
                (devID, name, datetime.now(), self.T, self.H))
        # commit changes
        con.commit()
        # close db
        con.close()
```

```
        # schedule the next task
❻       self.task = Timer(self.SCAN_INT, self.scan_task)
❼       self.task.start()
```

This method harnesses methods you've already defined. First you start the BlueZ tools by calling `start_scan()` ❶. Then you call `parse_hcidump()` to parse the advertisement data ❷, storing the retrieved values in a tuple. Right after that, you correct for the temperature offset added in the IoT device by subtracting `40`. (Recall that this offset was to accommodate negative temperature values.) Calling `stop_scan()` stops the BLE scanning ❸, and calling `send_alert()` sends an IFTTT alert if required ❹.

Next, you establish a connection to your SQLite database, allowing you to insert a row of values consisting of the MAC ID, device name, current time, temperature, and humidity level ❺. You then create a `Timer` object from the `threading` module and set it up to call the same `scan_task()` method after a time interval `SCAN_INT` ❻. Finally, you start the timer ❼. This way, once the time interval has passed, `scan_task()` will be executed in a new thread that will run parallel to the rest of the program, and the cycle will be repeated.

The Web Server Code

In this section, we'll look at the code in *server.py*, which implements a web server on the Raspberry Pi using `Bottle`. This code will generate a web page displaying the latest sensor values, as well as a plot of the data. Along with Python, you'll be using small doses of HTML, CSS, and JavaScript in your code, but you don't need to be an expert in web development to understand the project. At a high level, HTML provides *structure* for a web page, CSS determines the *style* of presentation, and JavaScript facilitates *actions* on the page.

To see the complete code listing, skip ahead to **"The Complete Python Web Server Code"** on **page 349**. You can also find this code at ***https://github.com/mkvenkit/pp2e/blob/main/iotgarden/server.py***.

Creating and Running the Server

The Python code to manage the web server is encapsulated in a class called `IOTGServer`. Here's the class's constructor:

```
class IOTGServer:

    def __init__(self, dbname, host, port):

        self._dbname = dbname

        self._host = host

        self._port = port

        # create bottle object

    ❶ self._app = Bottle()
```

The `IOTGServer` constructor takes in and stores the database name, hostname, and port number. The constructor also creates a `Bottle` instance ❶, which you'll use to implement the web server.

We briefly explored how the `Bottle` web framework works earlier in the chapter. As you saw, using `Bottle` involves defining routes to web resources and binding those routes to handler functions that will be called when someone visits that route. The `IOTGServer` class's `run()` method does just that.

```
def run(self):

    # ----------
```

```
    # add routes:
    # ----------
    # T/H data
❶   self._app.route('/thdata')(self.thdata)
    # plot image
❷   self._app.route('/image/<macid>')(self.plot_image)
    # static files - CSS, JavaScript
❸   self._app.route('/static/<filename>')(self.st_file)
    # main HTML page
❹   self._app.route('/')(self.main_page)
    # start server
❺   self._app.run(host=self._host, port=self._port)
```

You start by creating four routes, pairing each with its own handler method. The `/thdata` route ❶ returns the latest sensor data for all scanned devices in JSON format. The `/image/<macid>` route ❷ is for an image showing a plot of the sensor data. The plot image will be dynamically created from values in the SQLite database using `matplotlib`. The `<macid>` portion of the route uses `Bottle` URL template syntax to create a placeholder for your device's MAC ID. As you'll see later, the actual ID will be filled in by the JavaScript code. The route at ❸ is a little different: `Bottle` allows you to serve *static* files (files that you already have on disk) using the `/static` keyword in the route. In this case, you'll be serving JavaScript and CSS files using this scheme. Finally, the `/` route ❹ is for the main HTML page, which

will be loaded when you run the server. The method ends with a call to the `run()` method on the `Bottle` instance to start the server ❺.

Serving the Main Page

Now we'll look at `main_page()`, the handler method bound to the `Bottle` route for the main HTML page. This method will be called when the user navigates to *http://<iotgarden>.local:8080/* in a web browser.

```
    def main_page(self):
        """main HTML page"""
    ❶ response.content_type = 'text/html'
        strHTML = """
<!DOCTYPE html>
<html>
<head>
❷ <link href="static/style.css" rel="stylesheet">
❸ <script src="static/server.js"></script>
</head>
<body>
<div id = "title">The IoT Garden </div>
<hr/>
❹ <div id="sensors"></div>
```

```
</body>

</html>"""

    return strHTML
```

You set the content type of the response the method will return to be either text or HTML ❶. Then you put together the HTML as a multiline string declared within triple quotes (`"""`). In the HTML code, you load a CSS style sheet file ❷ and JavaScript file ❸. These files, which will help style the page and fetch the latest sensor data, will be served using the `/static` route we discussed earlier. You next declare an empty `<div>` element ❹, which is a section or division in an HTML document, assigning it an ID of `sensors`. This will be populated dynamically by the code in the JavaScript file, as you'll see later.

Here's the `/static` route's handler method, which serves the JavaScript and CSS files for the main page:

```
def st_file(self, filename):

  """serves static files"""

  return static_file(filename, root='./static')
```

The JavaScript and CSS files are in a */static* subfolder with respect to the Python code. You use the `Bottle` framework's `static_file()` method to serve the file from this subfolder with the given filename.

Retrieving the Sensor Data

As you've seen, there are two `Bottle` routes associated with sensor data: `/image/<macid>`, which retrieves a plot of a device's data, and `/thdata`, which retrieves the most recent sensor data for all devices. We'll look at the handler methods associated with those routes now,

starting with the `plot_image()` method, which is bound to the `/image/<macid>` route.

```
def plot_image(self, macid):
    """create a plot of sensor data by reading database"""
    # get data
❶   data = self.get_data(macid)
    # create plot
    plt.legend(['T', 'H'], loc='upper left')
❷   plt.plot(data)
    # save to a buffer
❸   buf = io.BytesIO()
❹   plt.savefig(buf, format='png')
    # reset stream position to start
    buf.seek(0)
    # read image data as bytes
❺   img_data = buf.read()
    # set response type
    response.content_type = 'image/png'
    # return image data as bytes
```

❻ return img_data

The method takes in the MAC ID of the device whose sensor data you want to plot. You call the `get_data()` helper method ❶, which we'll look at next, to retrieve that device's data from the SQLite database. The method returns a list of tuples with temperature and humidity readings in the form `[(T, H), (T, H), ...]`. You use `matplotlib` to plot this data ❷.

Normally, you'd call `plt.show()` to display a `matplotlib` plot on your computer, but in this case, your web server needs to send this data out as image bytes so the plot can be viewed in a browser. You use Python's `io.BytesIO` module to create a buffer that will act as a file stream to hold the image data ❸ and then save the plot to buffer in the PNG format ❹. Next, you reset the stream with `buf.seek(0)`, which in turn sets you up to read the image bytes from the beginning ❺. After setting the response return type to a PNG image, you return the image bytes ❻.

Here's the `get_data()` method that you called as part of `plot_image()`. It retrieves all temperature and humidity readings from the device with the given MAC ID.

```
def get_data(self, macid):

    # connect to database

    con = sqlite3.connect(self._dbname)

    cur = con.cursor()

    data = []

❶ for row in cur.execute("SELECT * FROM iotgarden_data

                where DEVID = :dev_id LIMIT 100", {"dev_id" : macid}):
```

```
    ❷ data.append((row[3], row[4]))

    # commit changes

    con.commit()

    # close db

    con.close()

    return data
```

After establishing a connection to your SQLite database, you issue a query to get the 100 most recent rows in the database with a `DEVID` equal to the MAC ID passed into this method ❶. The rows returned from the database have fields in the form `(DEVID, NAME, TS, T, H)`. You pick up just the last two elements of each row (a temperature reading and a humidity reading) and append them as a tuple to the `data` list ❷. You end up with the list of tuples that the `plot_image()` method expects.

The other handler method that works with sensor data is `thdata()`, the handler for the `/thdata` route. This method returns the latest temperature and humidity values for each of your Adafruit BLE peripheral devices:

```
def thdata(self):

    """connect to database and retrieve latest sensor data"""

    # connect to database

    con = sqlite3.connect(self._dbname)

    cur = con.cursor()
```

```
    macid = ""

    name = ""

    # set up a device list

    devices = []

    # get unique device list from db

  ❶ devid_list = cur.execute("SELECT DISTINCT DEVID FROM
iotgarden_data")

    for devid in devid_list:

      ❷ for row in cur.execute("SELECT * FROM iotgarden_data

           where DEVID = :devid ORDER BY TS DESC LIMIT 1",

           {"devid" : devid[0]}):

         ❸ devices.append({'macid': macid, 'name': name, 'T' : T, 'H': H})

       # commit changes

    con.commit()

    # close db

    con.close()

    # return device dictionary

  ❹ return {"devices" : devices}
```

Here you run the following query on your SQLite database: `SELECT DISTINCT DEVID FROM iotgarden_data` ❶. This returns all the unique

device IDs in the database. For example, if you have three Adafruit boards set up for this project, the query will return the device IDs for all three of them. For each device ID you've found, you run the following database query: `SELECT * FROM iotgarden_data where DEVID = :devid ORDER BY TS DESC LIMIT 1` ❷. This gets you the latest (by timestamp) row of data available for the given device ID. You add the retrieved information into a `devices` list ❸. Each element in the list is a dictionary. This list is mapped to a `"devices"` key in a dictionary and returned ❹. The format followed here is JSON—a nested dictionary of lists and dictionaries—which will be convenient to parse in the JavaScript code.

The JavaScript

Next, let's take a look at the JavaScript code in file *static/server.js*. This file is included in the HTML returned by the `main_page()` handler method, so the JavaScript code will run locally in a web browser on the user's machine (typically not the Raspberry Pi) when they visit the main project page. The code uses `Bottle` paths to dynamically add the sensor data to the home page's HTML.

```
// async function that fetches data from server

async function fetch_data() {

  ❶ let response = await fetch('thdata');

  ❷ devices_json = await response.json();

    console.log('updating HTML...');

  ❸ devices = devices_json["devices"];

  ❹ let strHTML = "";

  ❺ var ts = new Date().getTime();
```

```
❻ for (let i = 0; i < devices.length; i++) {
    // console.log(devices[i].macid)
    strHTML = '<div class="thdata">';
    strHTML += '<span>' + devices[i].name + '(' + devices[i].macid
      + '): </span>';
    strHTML += '<span>T = ' + devices[i].T +
          ' C (' + (9.0*devices[i].T/5.0 + 32.0) + ' F),</span>';
    strHTML += '<span> H = ' + devices[i].H + ' % </span>';
    strHTML += '</div>'; // thdata
    // create image div
  ❼ strHTML += '<div class="imdiv"><img src="image/' +
      devices[i]["macid"] + '?ts=' + ts + '"></div>';
    // add divider
    strHTML += '<hr/>';
  }
  // set HTML data
❽ document.getElementById("sensors").innerHTML = strHTML;
};
```

You first define a JavaScript function called `fetch_data()`. The `async` keyword in the definition indicates that you can call the `await` method from this function. `async` and `await` are modern JavaScript features that facilitate asynchronous programming. An example of asynchronous programming is when you request data from a server over a network, as you'll be doing here. You don't know when you'll get a response from the server, and you don't want to wait around for it. With `async` and `await`, you can go off and do other things and get notified when the response arrives. But when you get the response, it could be useful data, or it could indicate an error.

You make an asynchronous call using `await` to get the `/thdata` resource using the JavaScript `fetch()` method ❶. When this call reaches the server, it will end up calling the route handler `thdata()` in the `IOTGarden` class in *server.py*. You then make another asynchronous call ❷ to get the response from the call ❶. This call will return only when the response is received. You expect that response to be JSON data, and you retrieve the contents of the `"devices"` key from the data ❸. This will be a list of sensor data values from your devices.

Next, you create an empty string that you'll use to build up the HTML for showing the sensor data ❹. You use the JavaScript `Date()` method to get a current timestamp ❺, which you'll soon put to use in a little trick to ensure that the plot image loads correctly. Then you loop through all the devices ❻ and create the required HTML. Notice especially how you create an HTML `<img>` element for displaying the `matplotlib` plot ❼, supplying the `Bottle` route `/image/<macid>` as the location from which the image should be retrieved.

Once you've built up the HTML string, you add it to the home page's HTML ❽. Specifically, you set the HTML into the `<div>` with the ID of `"sensors"`. This was the empty `<div>` you created in `main_page()`, the `IOTGarden` class's handler method for the main page route in the server.

To better illustrate what the code in the `for` loop at ❻ is doing, here's an example of the output HTML string produced in one iteration through the loop. Note that the output has been formatted for readability.

```
❶ <div class="thdata">

   <span>IGDE74(DE:74:03:D9:3D:8B): </span>

   <span>T = 26 C, (73.4 F),</span>

   <span>H = 55 % </span>

</div>

❷ <div class="imdiv">

  ❸ <img src="image/DE:74:03:D9:3D:8B?ts=1635673486192">

</div>

❹ <hr>
```

You first have a `<div>` of class `thdata` ❶, which contains three `<span>` elements holding the device name and MAC ID, the latest temperature reading, and the latest humidity reading. Then you have another `<div>` of class `imdiv` ❷. This `<div>` contains the `<img>` element for displaying the plot ❸. The `src` of this element is set as `image/DE:74:03:D9:3D:8B?ts=1635673486192`. The first part of this, `image/DE:74:03:D9:3D:8B`, is the route to the `plot_image()` method in `IOTGarden`, which takes the device's MAC ID as an argument. The `ts` part is a current timestamp, which tricks the browser into not caching the image. Browsers use caching to skip loading web resources that they think they've already loaded recently, but you want to ensure that the plot image gets updated regularly. Adding the time-

stamp to the image's URL makes the URL different every time, so the browser will keep retrieving this resource.

The HTML string ends with a horizontal line ❹ to act as a separator between data for each device. If you have multiple Adafruit BLE devices, you'd see a similar block of HTML for the next device after this first one.

The *static/server.js* file concludes with this JavaScript code:

```
// fetch once on load

❶ window.onload = function() {

   fetch_data();

};

// now fetch data every 30 seconds

❷ setInterval(fetch_data, 30000)
```

Here you set an anonymous function to be called as soon as the web page loads ❶. This function will call `fetch_data()`, the asynchronous function you defined earlier. This ensures that sensor data will be displayed immediately when the user visits the page. You use the JavaScript `setInterval()` function ❷ to call `fetch_data()` every 30,000 milliseconds—or every 30 seconds, that is. This way the user will see real-time updates to both the plot and the latest sensor readings.

The CSS

CSS controls the appearance of a web page through a *style sheet*, which sets rules for how different HTML elements should be rendered. These rules rely on a *box model*, where every element in your

HTML is considered a rectangular box. You can control almost every aspect of a box's appearance by specifying colors, transparency, margins, borders, text fonts, layout qualifiers, and so on. The CSS rules for the project's main page are in the file *static/style.css*.

```
html {
    background-color: gray;
}
body {
    min-height: 100vh;
    max-width: 800px;
    background-color: #444444;
    margin-left: auto;
    margin-right: auto;
    margin-top: 0;
}
h1 {
    color: #aaaaaa;
    font-family: "Times New Roman", Times, serif;
}
#title {
```

```
    font-family: "Times New Roman", Times, serif;
    font-size: 40px;
    text-align: center;
}
❶ .thdata {
    display: flex;
    justify-content: center;
    width: 80%;
    color: #aaaaaa;
    font-family: Arial, Helvetica, sans-serif;
    font-size: 24px;
    margin: auto;
}
❷ .imdiv {
    display: flex;
    justify-content: center;
}
```

We won't dwell on the details of this CSS file, but notice the code blocks at ❶ and ❷. These lay out rules for the display of HTML elements with a `class` attribute of `thdata` and `imdiv`, respectively.

These are `<div>` elements generated by the JavaScript file we just looked at.

The Main Program File

The main program file *iotgarden.py* coordinates all the code running on the Raspberry Pi. This file is responsible for creating and managing the SQLite database, starting the BLE scanner, running the `Bottle` web server, and accepting command line arguments. To see the complete code listing, skip ahead to **"The Complete Main Program Code"** on **page 351**. You can also find this code at ***https://github.com/mkvenkit/pp2e/blob/main/iotgarden/iotgarden.py***.

The Database Setup

The main program uses function `setup_db()` to prepare the SQLite database. You'll call this function the first time you run the code, or anytime you want to clear the database of old data and start fresh.

```
def setup_db(dbname):
    """set up the database"""
    # connect to database - will create new if needed
❶   con = sqlite3.connect(dbname)
    cur = con.cursor()
    # drop if table exists
❷   cur.execute("DROP TABLE IF EXISTS iotgarden_data")
    # create table
```

```
❸ cur.execute("CREATE TABLE iotgarden_data(DEVID TEXT, NAME TEXT,
                TS DATETIME, T NUMERIC, H NUMERIC)")
```

You start by connecting to the SQLite database with a given name and path ❶. If the database doesn't exist yet, this call will create it. You clear off the `iotgarden_data` table if it already exists in the database ❷. Then you create a new table of that name ❸. The table is given the following fields: `DEVID` (a text field for the device's MAC ID), `NAME` (a text field for the name of the device), `TS` (a timestamp of type `DATETIME`), and numeric `T` (temperature) and `H` (humidity) fields.

The main program also has a utility function `print_db()` for listing the current contents of the database. This can be useful for debugging purposes, or if you want to view all the sensor data as simple text output rather than a graphical plot.

```
def print_db(dbname):
    """prints contents of database"""
    # connect to database
    con = sqlite3.connect(dbname)
    cur = con.cursor()
❶ for row in cur.execute("SELECT * FROM iotgarden_data"):
        print(row)
```

After connecting to the database, you execute a query to gather all rows from the table of sensor data ❶, which you then print out, one row at a time.

The main() Function

Now let's look at the `main()` function:

```
def main():
    print("starting iotgarden...")
    # set up cmd line argument parser
    parser = argparse.ArgumentParser(description="iotgarden.")
    # add arguments
  ❶ parser.add_argument('--createdb', action='store_true',
required=False)
  ❷ parser.add_argument('--lsdb', action='store_true', required=False)
  ❸ parser.add_argument('--hostname', dest='hostname',
required=False)
    args = parser.parse_args()
    # set database name
  ❹ dbname = 'iotgarden.db'
    if (args.createdb):
        print("Setting up database...")
      ❺ setup_db(dbname)
        print("done. exiting.")
        exit(0)
```

```
    if (args.lsdb):
        print("Listing database contents...")
    ❻ print_db(dbname)
        print("done. exiting.")
        exit(0)
    # set hostname
❼ hostname = 'iotgarden.local'
    if (args.hostname):
        hostname = args.hostname
    # create BLE scanner
❽ bs = BLEScanner(dbname)
    # start BLE
    bs.start()
    # create server
❾ server = IOTGServer(dbname, hostname, 8080)
    # run server
    server.run()
```

You use a `parser` object to add command line options for creating or resetting the database ❶ and for printing out the database contents ❷. You also add a `--hostname` option ❸, which lets you use a different

hostname—this is useful if you name your Pi something other than `iotgarden`.

Next, you declare the filename for the SQLite database ❹. Then, if the `--createdb` command line option was used, you call the `setup_db()` function discussed earlier ❺. You print out the database contents if the `--lsdb` command line option is set ❻. You then set the hostname to `iotgarden.local` by default ❼, but you override this if a different hostname was set in the command line.

To finish, you create an object of your `BLEScanner` class ❽ and set it in motion with its `start()` method. Similarly, you create the `Bottle` server ❾ and launch it by calling the `run()` method. The scanner and web server will run in parallel with each other.

Running the IoT Garden

The code for this project lives in two places. First, there's the CircuitPython code in *ble_sensors.py*, which needs to be renamed *code.py* and uploaded to the Adafruit BLE Sense boards, as discussed in **"CircuitPython Setup"** on **page 318**. Once you get each BLE board up and running, you'll need to determine its MAC address. For this, connect the board to power, and run the following on your Raspberry Pi:

```
$ sudo hcitool lescan
```

Here's my output as an example:

```
LE Scan...

57:E0:F5:93:AD:B1 (unknown)

57:E0:F5:93:AD:B1 (unknown)
```

DE:74:03:D9:3D:8B (unknown)

DE:74:03:D9:3D:8B IOTG1

27:FE:36:49:F0:2E (unknown)

7A:17:EB:3C:04:A5 (unknown)

7A:17:EB:3C:04:A5 (unknown)

Here, my board's MAC ID `DE:74:03:D9:3D:8B` appears next to `IOTG1`. (It will likely be a different ID for you.) Take note of this ID and add it to the allowlist in your BLE scanner code.

The rest of the code goes on the Raspberry Pi, which you can work with using SSH and VS Code, as discussed in **Appendix B**. When you're ready to try it, run the following from the code directory:

```
$ sudo python iotgarden.py
```

You'll see a stream of messages on your shell similar to the following output:

---

starting iotgarden...

BLE scan started...

CompletedProcess(args=['sudo', 'hciconfig', 'hci0', 'reset'], returncode=0)

04 3E 1C 02 01 02 01 8B 3D D9 03 74 DE 10 0F FF 22 08 0B CD AB 49 4F 54 47 31

1A 3A 30 30 C9 26 58

BLE scan stopped.

```
Bottle v0.12.19 server starting up (using WSGIRefServer())...

Listening on http://iotgarden.local:8080/

Hit Ctrl-C to quit.
```

Now, open a browser window on any computer on the same local network, and navigate to *http://<iotgarden>.local:8080/*, substituting your Raspberry Pi name as appropriate. You should see output similar to **Figure 14-4**.

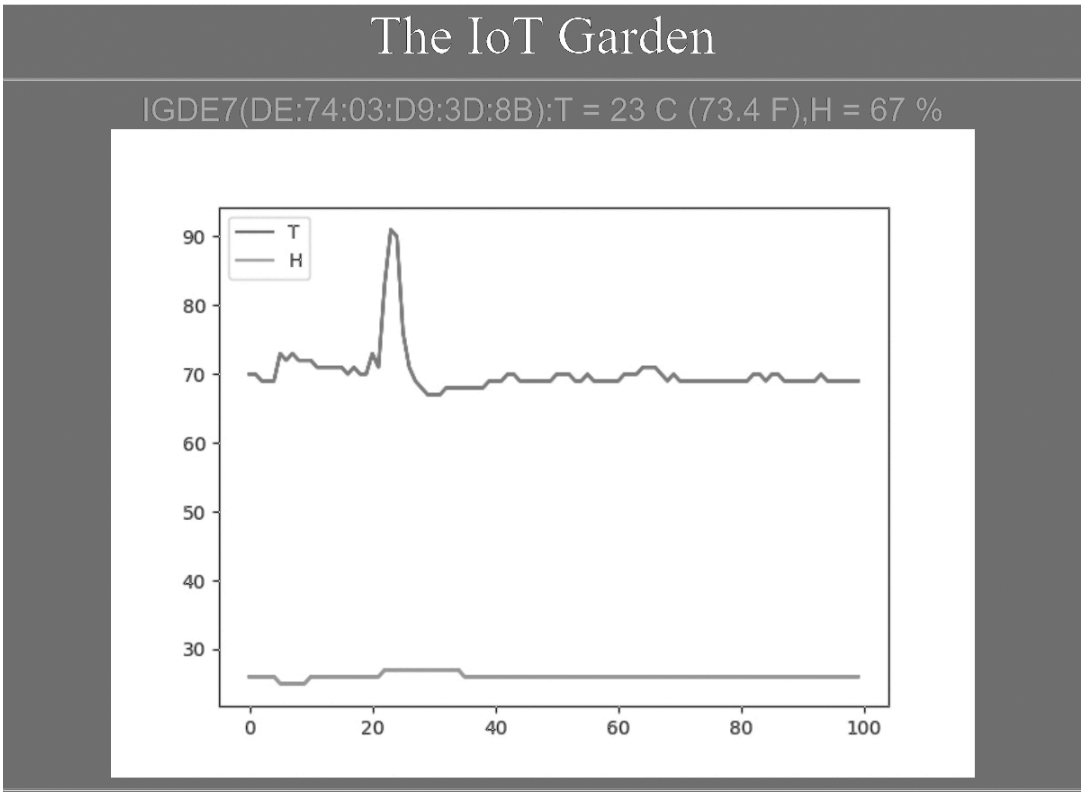

Figure 14-4: *The browser output from the IoT garden project*

The browser output will show a web page with the device ID and the latest temperature and humidity values, along with a graph of the latest 100 values from the device. This information will be repeated for each device that you have configured. The graph will refresh automatically every 30 seconds to show you the latest data.

If you don't want to wait for an exceptionally hot or dry day to test the IFTTT alert system, put your finger on the temperature sensor (or warm it up slightly using any other method, taking care not to damage the board) so that it exceed the temperature threshold `self.TMAX` set in *BLEScanner.py*. You should see a message like this in your shell on the Raspberry Pi:

Triggering IFTTT alert!

In a few seconds, you should also get an alert on the IFTTT app on your phone.

Summary

We've covered a lot of ground in this chapter! You learned about a multitier IoT architecture consisting of hardware, software, and the cloud. You learned how to use CircuitPython to get data from sensors and transmit it via the BLE wireless protocol. You also learned how to run a simple web server on your Raspberry Pi and how to display sensor data in a web browser using HTML, JavaScript, and CSS. You even learned how to get live alerts from your IoT garden using the IFTTT service.

Experiments!

1. 1. For simplicity, the web page you created to display your sensor data is quite limited, but it provides a good framework you can build on to create more sophisticated visualizations of the data, especially since you're using a structured database like SQLite to store the sensor values. Here are a few suggestions to improve the page:
    1. Instead of showing data from all the BLE Sense boards at once, create a pull-down menu to select the board whose data you want to display.

2. Implement a way to set how many days of data you want to show in the plot. You can use the JavaScript DatePicker, along with a custom `Bottle` route and a special SQLite query to implement this.
3. Customize your JavaScript code to update the latest sensor values every few seconds and only refresh the plot on a longer time scale.

2. You wrote the CircuitPython code to transmit the temperature and humidity data as single-byte integers, but what if you want more accuracy? How could you transmit a value like `26.54` over BLE? (Hint: there are two unused bytes in the manufacturer data field of the BLE packet. For a value like `26.54`, you could store the `54` in a byte and divide it by `100` upon receipt in the BLE scanner code.)
3. Power consumption is a major concern in IoT. Think about how you can save battery power on the devices. One method is to slow down the BLE advertisement interval. Another option you can explore is to use the Arduino C++ library instead of CircuitPython on the Adafruit boards. This will allow you to reduce the power consumption of the device by putting it to deep sleep during times when it isn't reading the sensor data or transmitting BLE messages.

The Complete CircuitPython Code

Here's the complete *ble_sensors.py* code listing:

```
"""

ble_sensors.py

CircuitPython code for Adafruit BLE Sense boards. This program reads

data from the built-in Temperature and Humidity sensors, and puts
the

data in the BLE advertisement packet.
```

```
Author: Mahesh Venkitachalam

"""

import time

import struct

import board

import adafruit_bmp280

import adafruit_sht31d

from adafruit_ble import BLERadio

from adafruit_ble.advertising import Advertisement, LazyObjectField

from adafruit_ble.advertising.standard import ManufacturerData,
ManufacturerDataField

import _bleio

import neopixel

# derived from adafruit_ble class Advertisement

class IOTGAdvertisement(Advertisement):

    flags = None

    match_prefixes = (

        struct.pack(

            "<BHBH",  # prefix format
```

```
            0xFF,     # 0xFF is "Manufacturer Specific Data" as per BLE spec
            0x0822,   # 2-byte company ID
            struct.calcsize("<H9s"), # data format
            0xabcd # our ID
        ), # comma required - a tuple is expected
    )
    manufacturer_data = LazyObjectField(
        ManufacturerData,
        "manufacturer_data",
        advertising_data_type=0xFF, # 0xFF is "Manufacturer Specific
Data" as per BLE spec
        company_id=0x0822,          # 2-byte company ID
        key_encoding="<H",
    )
    # set manufacturer data field
    md_field = ManufacturerDataField(0xabcd, "<9s")
def main():
    # initialize I2C
    i2c = board.I2C()
```

```
    # initialize sensors

    bmp280 = adafruit_bmp280.Adafruit_BMP280_I2C(i2c)

    sht31d = adafruit_sht31d.SHT31D(i2c)

    # initialize BLE

    ble = BLERadio()

    # create custom advertisement object

    advertisement = IOTGAdvertisement()

    # append first 2 hex bytes (4 characters) of MAC address to name

    addr_bytes = _bleio.adapter.address.address_bytes

    name = "{0:02x}{1:02x}".format(addr_bytes[5],
addr_bytes[4]).upper()

    # set device name

    ble.name = "IG" + name

    # set initial value

    # will use only first 5 chars of name

    advertisement.md_field = ble.name[:5] + "0000"

    # BLE advertising interval in seconds

    BLE_ADV_INT = 0.2

    # start BLE advertising
```

```
ble.start_advertising(advertisement, interval=BLE_ADV_INT)

# set up NeoPixels and turn them all off

pixels = neopixel.NeoPixel(board.NEOPIXEL, 1,

                           brightness=0.1, auto_write=False)

# main loop

while True:

    # print values - this will be available on serial

    print("Temperature: {:.1f} C".format(bmp280.temperature))

    print("Humidity: {:.1f} %".format(sht31d.relative_humidity))

    # get sensor data

    # BMP280 range is -40 to 85 deg C, so add an offset to support

    # negative temperatures

    T = int(bmp280.temperature) + 40

    H = int(sht31d.relative_humidity)

    # stop advertising

    ble.stop_advertising()

    # update advertisement data

    advertisement.md_field = ble.name[:5] + chr(T) + chr(H) + "00"

    # start advertising
```

```
        ble.start_advertising(advertisement, interval=BLE_ADV_INT)

        # blink neopixel LED

        pixels.fill((255, 255, 0))

        pixels.show()

        time.sleep(0.1)

        pixels.fill((0, 0, 0))

        pixels.show()

        # sleep for 2 seconds

        time.sleep(2)

# call main

if __name__ == "__main__":

    main()
```

The Complete BLE Scanner Code

Here's the complete listing for the BLE scanner code, in file *BLEScanner.py*:

```
"""

BLEScanner.py

This class uses BlueZ hciconfig, hcitool, and hcidump tools to parse
```

advertisement data from BLE peripherals. It then stores them in a database.

This class also sends alerts via the IFTTT service.

Author: Mahesh Venkitachalam

"""

```
import sqlite3

import subprocess

from threading import Timer

import sys

import os

import time

import requests

from datetime import datetime

class BLEScanner:

    def __init__(self, dbname):

        """BLEScanner constructor"""

        self.T = 0

        self.H = 0

        # max values
```

```
        self.TMAX = 30
        self.HMIN = 20
        # time stamp for last alert
        self.last_alert = time.time()
        # alert interval in seconds
        self.ALERT_INT = 60
        # scan interval in seconds
        self.SCAN_INT = 10
        self._dbname = dbname
        self.hcitool = None
        self.hcidump = None
        self.task = None
        # ------------------------------------------
        # peripheral allow list - add your devices here!
        # ------------------------------------------
        self.allowlist = ["DE:74:03:D9:3D:8B"]
    def start(self):
        """start BLE scan"""
        # start task
```

```
        self.scan_task()

    def stop(self):

        """stop BLE scan"""

        # stop timer

        self.task.cancel()

    def send_alert(self):

        """send IFTTT alert if sensor data has exceeded the thresholds"""

        # check T, H

        delta = time.time() - self.last_alert

        # print("delta: ", delta)

        if ((self.T > self.TMAX) or (self.H < self.HMIN)) and (delta >
self.ALERT_INT):

            print("Triggering IFTTT alert!")

            key = '6zmfaOBei1DgdmlOgOi6C' # USE YOUR KEY HERE!

            url = 'https://maker.ifttt.com/trigger/TH_alert/json/with/key/' +
key

            json_data = {"T": self.T, "H": self.H}

            r = requests.post(url, data = json_data)

            # save last alert

            self.last_alert = time.time()
```

```
    def start_scan(self):
        """starts the BlueZ tools required for scanning"""
        print("BLE scan started...")
        # reset device
        ret = subprocess.run(['sudo', '-n', 'hciconfig', 'hci0', 'reset'],
                             stdout=subprocess.DEVNULL)
        print(ret)
        # start hcitool process
        self.hcitool = subprocess.Popen(['sudo', '-n', 'hcitool', 'lescan', '--
duplicates'],
                                        stdout=subprocess.DEVNULL)
        # start hcidump process
        self.hcidump = subprocess.Popen(['sudo', '-n', 'hcidump', '--raw'],
                                        stdout=subprocess.PIPE)
    def stop_scan(self):
        """stops BLE scan by killing BlueZ tools processes."""
        subprocess.run(['sudo', 'kill', str(self.hcidump.pid), '-s', 'SIGINT'])
        subprocess.run(['sudo', '-n', 'kill', str(self.hcitool.pid), '-s', "SIGINT"])
        print("BLE scan stopped.")
```

```
    def parse_data(self, data):

        """parses hcdump string and outputs MACID, name, manufacturer
data"""

        fields = {}

        # parse MACID

        x = [int(val, 16) for val in data.split()]

        macid = ":".join([format(val, '02x').upper() for val in x[7:13][::-1]])

        # check if MACID is in allowlist

        if macid in self.allowlist:

            # look at 6th byte to see PDU type

            if (x[5] == 0x02): # ADV_IND

                print(data)

                fields["macid"] = macid

                # set pkt type

                #fields["ptype"] = "ADV_IND"

                # parse data

                fields["T"] = x[26]

                fields["H"] = x[27]

                name = "".join([format(val, '02x').upper() for val in x[21:26]])
```

```
            name = bytearray.fromhex(name).decode()
            fields["name"] = name
        return fields
    def parse_hcidump(self):
        """parse output from hcidump"""
        data = ""
        (macid, name, T, H) = (None, None, None, None)
        while True:
            line = self.hcidump.stdout.readline()
            line = line.decode()
            if line.startswith('> '):
                data = line[2:]
            elif line.startswith('< '):
                data = ""
            else:
                if data:
                    # concatenate lines
                    data += line
                    # a tricky way to remove whitespace
```

```
                data = " ".join(data.split())

                # parse data

                fields = self.parse_data(data)

                success = False

                try:

                    macid = fields["macid"]

                    T = fields["T"]

                    H = fields["H"]

                    name = fields["name"]

                    success = True

                except KeyError:

                    # skip this error, since this indicates

                    # invalid data

                    pass

                if success:

                    return (macid, name, T, H)

    def scan_task(self):

        """the scanning task which is run on a separate thread"""

        # start BLE scan
```

```
self.start_scan()

# get data

(macid, name, self.T, self.H) = self.parse_hcidump()

# correct temperature offset

self.T = self.T - 40

print(self.T, self.H)

# stop BLE scan

self.stop_scan()

# send alert if required

self.send_alert()

# write to db

# connect to database

con = sqlite3.connect(self._dbname)

cur = con.cursor()

devID = macid

# add data

with con:

    cur.execute("INSERT INTO iotgarden_data VALUES (?, ?, ?, ?, ?)",

        (devID, name, datetime.now(), self.T, self.H))
```

```
        # commit changes
        con.commit()
        # close db
        con.close()
        # schedule the next task
        self.task = Timer(self.SCAN_INT, self.scan_task)
        self.task.start()
# use this for testing the class independently
def main():
    print("starting BLEScanner...")
    bs = BLEScanner("iotgarden.db")
    bs.start()
    data = None
    while True:
        try:
            (macid, name, T, H) = bs.parse_hcidump()
            # exit(0)
        except:
            bs.stop()
```

```
            print("stopped. Exiting")
            exit(0)
        print(macid, name, T, H)
        time.sleep(10)
if __name__ == '__main__':
    main()
```

The Complete Python Web Server Code

Before we move on to the JavaScript and CSS files that round out the web server code, here's a complete listing of the Python portion of the web server code, in file *server.py*.

```
"""
server.py
This program creates a Bottle.py based web server. It also creates a
plot from the sensor data.
Author: Mahesh Venkitachalam
"""
from bottle import Bottle, route, template, response, static_file
from matplotlib import pyplot as plt
import io
```

```
import sqlite3

class IOTGServer:

    def __init__(self, dbname, host, port):

        """constructor for IGServer"""

        self._dbname = dbname

        self._host = host

        self._port = port

        # create bottle object

        self._app = Bottle()

    def get_data(self, macid):

        # connect to database

        con = sqlite3.connect(self._dbname)

        cur = con.cursor()

        data = []

        for row in cur.execute("SELECT * FROM iotgarden_data where
DEVID = :dev_id LIMIT 100",

                               {"dev_id" : macid}):

            data.append((row[3], row[4]))

        # commit changes
```

```
        con.commit()

        # close db

        con.close()

        return data

    def plot_image(self, macid):

        """create a plot of sensor data by reading database"""

        # get data

        data = self.get_data(macid)

        # create plot

        plt.legend(['T', 'H'], loc='upper left')

        plt.plot(data)

        # save to a buffer

        buf = io.BytesIO()

        plt.savefig(buf, format='png')

        # reset stream position to start

        buf.seek(0)

        # read image data as bytes

        img_data = buf.read()

        # set response type
```

```
        response.content_type = 'image/png'
        # return image data as bytes
        return img_data
    def main_page(self):
        """main HTML page"""
        response.content_type = 'text/html'
        strHTML = """
<!DOCTYPE html>
<html>
<head>
<link href="static/style.css" rel="stylesheet">
<script src="static/server.js"></script>
</head>
<body>
<div id = "title">The IoT Garden </div>
<hr/>
<div id="sensors"></div>
</body>
</html>"""
```

```
        return strHTML

    def thdata(self):

        """connect to database and retrieve latest sensor data"""

        # connect to database

        con = sqlite3.connect(self._dbname)

        cur = con.cursor()

        # set up a device list

        devices = []

        # get unique device list from db

        devid_list = cur.execute("SELECT DISTINCT DEVID FROM
iotgarden_data")

        # print(devid_list)

        for devid in devid_list:

            for row in cur.execute("SELECT * FROM iotgarden_data where
DEVID = :devid

                ORDER BY TS DESC LIMIT 1",

                {"devid" : devid[0]}):

                devices.append({'macid': row[0], 'name': row[1], 'T' : row[3],
'H': row[4]})

         # commit changes
```

```
    con.commit()
    # close db
    con.close()
    # return device dictionary
    return {"devices" : devices}
def st_file(self, filename):
    """serves static files"""
    return static_file(filename, root='./static')
def run(self):
    # ----------
    # add routes:
    # ----------
    # T/H data
    self._app.route('/thdata')(self.thdata)
    # plot image
    self._app.route('/image/<macid>')(self.plot_image)
    # static files - CSS, JavaScript
    self._app.route('/static/<filename>')(self.st_file)
    # main HTML page
```

```
        self._app.route('/')(self.main_page)

        # start server

        self._app.run(host=self._host, port=self._port)
```

The Complete Main Program Code

Here's a listing for the complete main program code in file *iotgarden.py*.

```
"""

iotgarden.py

Main program for the IoT Garden project. Sets up database, starts the
Bottle

web server, and the BLE scanner.

Author: Mahesh Venkitachalam

"""

import argparse

import sqlite3

from BLEScanner import BLEScanner

from server import IOTGServer

def print_db(dbname):

    """prints contents of database"""
```

```
    # connect to database
    con = sqlite3.connect(dbname)
    cur = con.cursor()
    for row in cur.execute("SELECT * FROM iotgarden_data"):
        print(row)
def setup_db(dbname):
    """set up the database"""
    # connect to database - will create new if needed
    con = sqlite3.connect(dbname)
    cur = con.cursor()
    # drop if table exists
    cur.execute("DROP TABLE IF EXISTS iotgarden_data")
    # create table
    cur.execute("CREATE TABLE iotgarden_data(DEVID TEXT, NAME
TEXT,
                TS DATETIME, T NUMERIC, H NUMERIC)")
def main():
    print("starting iotgarden...")
    # set up cmd line argument parser
```

```
    parser = argparse.ArgumentParser(description="iotgarden.")

    # add arguments

    parser.add_argument('--createdb', action='store_true',
required=False)

    parser.add_argument('--lsdb', action='store_true', required=False)

    parser.add_argument('--hostname', dest='hostname', required=False)

    args = parser.parse_args()

    # set database name

    dbname = 'iotgarden.db'

    if (args.createdb):

        print("Setting up database...")

        setup_db(dbname)

        print("done. exiting.")

        exit(0)

    if (args.lsdb):

        print("Listing database contents...")

        print_db(dbname)

        print("done. exiting.")

        exit(0)
```

```
    # set hostname
    hostname = 'iotgarden.local'
    if (args.hostname):
        hostname = args.hostname
    # create BLE scanner
    bs = BLEScanner(dbname)
    # start BLE
    bs.start()
    # create server
    server = IOTGServer(dbname, hostname, 8080)
    # run server
    server.run()
# call main
if __name__ == "__main__":
    main()
```
